1. Problem Title & Link

Title: LeetCode 205 – Isomorphic Strings

Link: https://leetcode.com/problems/isomorphic-strings/

2. Problem Statement (Short Summary)

Given two strings s and t, determine whether they are **isomorphic**.

Two strings are isomorphic if:

- Every character in s maps to exactly one character in t.
- No two different characters in s map to the same character in t.
- The order of characters must remain the same.

Return true if the strings are isomorphic; otherwise, return false.

3. Examples (Input → Output)

Example 1

Input

s = "egg"

t = "add"

Output

true

Explanation

e → a

g → d

The mapping is consistent.

Example 2

Input

s = "foo"

t = "bar"

Output

false

Explanation

o → a

o → r

One character maps to two different characters.

Example 3

Input

s = "badc"

t = "baba"

Output

false

Explanation

a → a

d → a

Two different characters map to the same character.

4. Constraints

- 1 <= s.length <= 5 × 10^4
- t.length == s.length
- Strings contain any valid ASCII character.

5. Core Concept (Pattern / Topic)

- HashMap
- Character Mapping
- Bidirectional Mapping

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every character:

- Search whether it was mapped before.
- Check whether another character already maps to the same target.

Repeated searching makes the solution inefficient.

Optimized Thinking

Maintain **two hash maps**.

s → t

and

t → s

For every character pair:

- If mapping already exists, verify it.
- Otherwise, create the mapping in both directions.

If any mismatch occurs, return false.

Intuition

Example

s = paper

t = title

Mappings

p → t

a → i

e → l

r → e

Every mapping is unique.

↓

Valid.

Why This Works

Using two hash maps guarantees:

- One character maps to only one character.
- Two different characters cannot map to the same character.

Thus, the mapping becomes one-to-one (bijective).

7. Visual / Intuition Diagram (ASCII)

```
s : p a p e r
    ↓ ↓ ↓ ↓ ↓
t : t i t l e
Maps
Forward

p → t
a → i
e → l
r → e

Backward

t → p
i → a
l → e
e → r
No conflicts
↓
Isomorphic
```

8. Pseudocode (Language Independent)

```
create mapST
create mapTS

for every index

    if s already mapped
        mapping must equal t

    if t already mapped
        mapping must equal s

    store mappings

return true
```

9. Code Implementation

Python

```python
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:

        s_to_t = {}
        t_to_s = {}

        for c1, c2 in zip(s, t):

            if c1 in s_to_t and s_to_t[c1] != c2:
                return False

            if c2 in t_to_s and t_to_s[c2] != c1:
                return False

            s_to_t[c1] = c2
            t_to_s[c2] = c1

        return True
```

Alternative (Pythonic One-Liner)

```python
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        return len(set(zip(s, t))) == len(set(s)) == len(set(t))
```

Why it works

- set(zip(s, t)) stores unique character mappings.
- len(set(s)) is the number of unique characters in s.
- len(set(t)) is the number of unique characters in t.

For an isomorphic mapping, all three counts must be equal.

Java

```
import java.util.HashMap;

class Solution {

    public boolean isIsomorphic(String s, String t) {

        HashMap<Character, Character> sToT = new HashMap<>();
        HashMap<Character, Character> tToS = new HashMap<>();

        for (int i = 0; i < s.length(); i++) {

            char c1 = s.charAt(i);
            char c2 = t.charAt(i);

            if (sToT.containsKey(c1) && sToT.get(c1) != c2)
                return false;

            if (tToS.containsKey(c2) && tToS.get(c2) != c1)
                return false;

            sToT.put(c1, c2);
            tToS.put(c2, c1);
        }

        return true;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

One pass through both strings.

Space Complexity

O(k)

- k = number of unique characters.
- Worst case: O(n).

11. Common Mistakes / Edge Cases

- Using only one hash map.
- Forgetting reverse mapping.
- Allowing two characters to map to the same character.

- Updating mappings before validation.
- Ignoring repeated characters.

12. Detailed Dry Run

Example

s = "paper"

t = "title"

i	s[i]	t[i]	s→t	t→s	Valid
0	p	t	p→t	t→p	✓
1	a	i	a→i	i→a	✓
2	p	t	Exists	Exists	✓
3	e	l	e→l	l→e	✓
4	r	e	r→e	e→r	✓

Output

true

13. Common Use Cases (Real-Life / Interview)

- Encoding and decoding systems
- Character substitution ciphers
- Dictionary mapping
- Data transformation
- Pattern matching

14. Common Traps (Important!)

- Forgetting the reverse map.
- Assuming equal frequencies imply isomorphism.
- Updating the map before checking existing mappings.
- Confusing isomorphic strings with anagrams.

15. Builds To (Related LeetCode Problems)

- LC 290 – Word Pattern
- LC 242 – Valid Anagram
- LC 890 – Find and Replace Pattern
- LC 49 – Group Anagrams
- LC 535 – Encode and Decode TinyURL

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force Search	$O(n^2)$	O(1)	Too slow
Two HashMaps	O(n)	O(k)	Standard interview solution
Python Set + Zip Trick	O(n)	O(k)	Elegant Python-only solution

17. Why This Solution Works (Short Intuition)

A valid isomorphic mapping must be **one-to-one**. Maintaining mappings in both directions ensures that each character in s maps to exactly one character in t, and no two characters in s map to the same character in t.

18. Variations / Follow-Up Questions

1. Solve using arrays instead of hash maps (ASCII characters).
2. Generalize the solution for Unicode strings.
3. Determine whether two arrays are isomorphic.
4. Return the actual character mapping instead of true/false.
5. Extend the problem to word-level mappings (see LC 290 – Word Pattern).
6.